

Embedded Systems Architectures - Laboratory Guide 8

Wi-Fi Connectivity

Academic Year 2025/2026

Authors: Arnaldo Oliveira, Fábio Coutinho

1. Objectives

- Familiarization with Wi-Fi operating modes on the ESP32-C6
- Using Station and SoftAP modes
- Deployment and testing of a basic HTTPS server with Wi-Fi connectivity

2. Introduction

Wi-Fi is a registered trademark of wireless communication technology owned by the Wi-Fi Alliance (WFA), the entity responsible for overseeing the Wi-Fi certification process. It is a family of wireless network protocols based on the IEEE 802.11 standards. Products that pass rigorous testing receive the Wi-Fi CERTIFIED™ seal, proving they meet industry-defined requirements for interoperability, security, and application-specific protocols. Compared to other wireless communication technologies, Wi-Fi offers greater coverage, better penetration through walls, and higher transmission rates.

The fundamental element of an IEEE 802.11 network is the Basic Service Set (BSS). This can be divided into two categories: Standalone BSS and Infrastructure BSS. In a BSS infrastructure, stations (STAs) must associate with an Access Point (AP) to obtain network services. Access Points (APs) function as the control center for the cluster, which is the most common network architecture. Each Access Support Terminal (STA) must go through the association and authorization processes before joining a given Business System Support System (BSS).

From a practical perspective, Wi-Fi network architectures are accompanied by their own identifiers:

- **BSS Identification (BSSID)** – each BSS has a physical identification address, called a BSSID. In an Infrastructure BSS, the BSSID corresponds to the MAC address of the Access Point (AP). This value is set by default, but can be changed following a specific naming format.
- **Service Set Identification (SSID)** – each AP also has an identifier intended for the user, called an SSID. In most cases, a BSSID is associated with a single SSID. This identifier is usually a human-readable text, commonly called the Wi-Fi network name.

Given this scope, it is important to understand how these same structures and identifiers apply to embedded devices that integrate wireless connectivity. In the case of the ESP32-C6, the ESP-IDF framework provides full Wi-Fi support according to IEEE 802.11 standards, allowing the SoC to function both as a Station (STA) - connecting to an existing Access Point - and as a SoftAP, creating its own network. The configuration of these functionalities is done through specific APIs that allow defining the SSID, security, channel, and managing association events, IP address acquisition, and loss of connectivity.

The integration of Wi-Fi in the ESP32-C6 follows exactly the principles described above: when acting as an STA, the device goes through the process of network search, authentication, and association with a BSSID; when acting as a SoftAP, the ESP itself announces the SSID, provides IP addresses via DHCP, and manages the stations that connect.

3. Practical Work

The practical work in this class is based on the ESP32-C6 Wi-Fi Reference API and the ESP32-C6 HTTPS Server Reference API, along with the ESP32-C6 Technical Reference Manual:

https://documentation.espressif.com/esp32-c6_technical_reference_manual_en.pdf

The Espressif documentation for the Wi-Fi Driver is required reading to understand the example code.

3.1 Wi-Fi Client/Station (STA) Mode

The Wi-Fi Client Mode example demonstrates how to configure the ESP32-C6 to operate in Station (STA) mode, connecting to an existing Wi-Fi access point, such as an Access Point (AP) created by the student's mobile phone. The program starts the network subsystem, attempts to connect to the network defined in **menuconfig**, and logs events to track the connection process, including reconnection attempts and failures. When the connection is successful, the device obtains an IPv4 address via DHCP from the AP, and this information is displayed on the terminal.

Question 1: In VSCode, create a new project from the *wifi -> getting_started -> station* template. Using the **menuconfig** command, set the SSID and password of the Wi-Fi network you configured on your phone. Access to this configuration in the project is found in the Example Configuration section of **menuconfig**. Compile the program and download the firmware for the SoC. You should observe the following event log:

```
...
I (355) main_task: Started on CPU0
I (365) main_task: Calling app_main()
I (375) wifi station: ESP_WIFI_MODE_STA
...
W (545) wifi:ACK_TAB0 :0x 90a0b, QAM16:0x9 (24Mbps), QPSK:0xa (12Mbps), BPSK:0xb (6Mbps)
W (545) wifi:CTS_TAB0 :0x 90a0b, QAM16:0x9 (24Mbps), QPSK:0xa (12Mbps), BPSK:0xb (6Mbps)
W (555) wifi:(agc)0x600a7128:0xd21aaa08, min.avgNF:0xce->0xd2 (dB), RCalCount:0x1aa,
min.RRssi:0xa08 (-95.50)
I (565) wifi:11ax coex: WDEVAX_PTIO(0x000a00a0), WDEVAX_PTII(0x000052bc).
```

```

I (565) wifi:mode : sta (10:51:db:01:3b:2c)
I (575) wifi:enable tsf
I (575) wifi station: wifi_init_sta finished.
...
I (895) wifi:(connect)dot11_authmode:0x3, pairwise_cipher:0x3, group_cipher:0x3
I (905) wifi:state: init -> auth (0xb0)
I (945) wifi:state: auth -> assoc (0x0)
...
I (1085) wifi:state: assoc -> run (0x10)
...
I (1145) wifi:connected with ASRO-Avr-AP, aid = 21, channel 6, BW20, bssid = 1e:21:b4:6f:e8:76
I (1155) wifi:security: WPA2-PSK, phy:ax, rssi:-30, cipher(pairwise:0x3, group:0x3), pmf:0,
I (1165) wifi:pm start, type: 1, twt_start:0

I (1165) wifi:pm start, type:1, aid:0x15, trans-BSSID:1e:21:b4:6f:e8:76, BSSID[5]:0x76,
mbssid(max-indicator:0, index:0), he:1
...
I (1225) wifi:AP's beacon interval = 102400 us, DTIM period = 1
...
W (2265) wifi:<ba-add>idx:0, ifx:0, tid:0, TAHI:0x10076e8, TALO:0x6fb4211e, (ssn:17, win:64,
cur_ssn:17), CONF:0xc0000005
I (27555) esp_netif_handlers: sta ip: 192.168.68.50, mask: 255.255.252.0, gw: 192.168.68.1
I (27555) wifi station: got ip:192.168.68.50
...

```

Question 2: Now that the ESP32-C6 has IPv4 connectivity, connect the PC to the same Access Point and test connectivity with the IoT device. The test should be successful, with latency reduced to a few milliseconds due to the fewer hops in the connection between the two peers.

Question 3: Cause a momentary disruption in the AP, for example, by temporarily turning it off for 2/3 seconds and restoring it to operation. You should observe behavior consistent with the following list:

```

I (88156) wifi:bcn_timeout,ap_probe_send_start
I (90656) wifi:ap_probe_send over, reset wifi status to disassoc
I (90656) wifi:state: run -> init (0xc800)
I (90666) wifi:pm stop, total sleep time: 13463547 us / 23811903 us
I (90666) wifi:<ba-del>idx:0, tid:6
I (90666) wifi:new:<6,0>, old:<6,0>, ap:<255,255>, sta:<6,0>, prof:1, snd_ch_cfg:0x0
I (90676) wifi station: retry to connect to the AP
I (90676) wifi station: connect to the AP fail
I (93086) wifi station: retry to connect to the AP
I (93086) wifi station: connect to the AP fail
...
I (93496) wifi:connected with phobar, aid = 1, channel 6, BW20, bssid = 52:6a:a3:22:c6:39
I (93506) wifi:security: WPA2-PSK, phy:ax, rssi:-34, cipher(pairwise:0x3, group:0x3), pmf:0,
I (93516) wifi:pm start, type: 1, twt_start:0
...
I (94836) esp_netif_handlers: sta ip: 192.168.68.50, mask: 255.255.255.240, gw: 192.168.68.1
I (94836) wifi station: got ip: 192.168.68.50

```

As you can see, the connection should be restored, and the IP address obtained again.

3.2 Wi-Fi “Soft” Access Point (SoftAP) Mode

In the SoftAP example, we will configure the ESP32-C6 to operate as a Wi-Fi Access Point. This means that the ESP itself creates and advertises an autonomous Wi-Fi network, allowing other devices (such as smartphones, laptops, or IoT modules) to connect directly to it, without the need for an external router.

The example includes a monitoring mechanism that prints log messages whenever a station (device) connects to the access point. In this context, the ESP records the device's MAC address, association identifier (AID), and other relevant information, allowing you to know exactly when and who joined the network.

Question 4: In VSCode, create a new project from the *wifi -> getting_started -> softAP* template. Using the **menuconfig** command, define the SSID and Password of the Wi-Fi network in the Example Configuration section. Compile the program and download the firmware to the SoC. You should observe the following event log:

```
...
I (364) main_task: Started on CPU0
I (374) main_task: Calling app_main()
I (394) wifi softAP: ESP_WIFI_MODE_AP
...
W (564) wifi:ACK_TAB0 :0x 90a0b, QAM16:0x9 (24Mbps), QPSK:0xa (12Mbps), BPSK:0xb (6Mbps)
W (564) wifi:CTS_TAB0 :0x 90a0b, QAM16:0x9 (24Mbps), QPSK:0xa (12Mbps), BPSK:0xb (6Mbps)
W (564) wifi:(agc)0x600a7128:0xd200e9ea, min.avgNF:0xce->0xd2 (dB), RCalCount:0xd,
min.RRssi:0x9ea(-97.38)
I (574) wifi:llax coex: WDEVAX_PTIO(0x000a00a0), WDEVAX_PTII(0x000062bc).

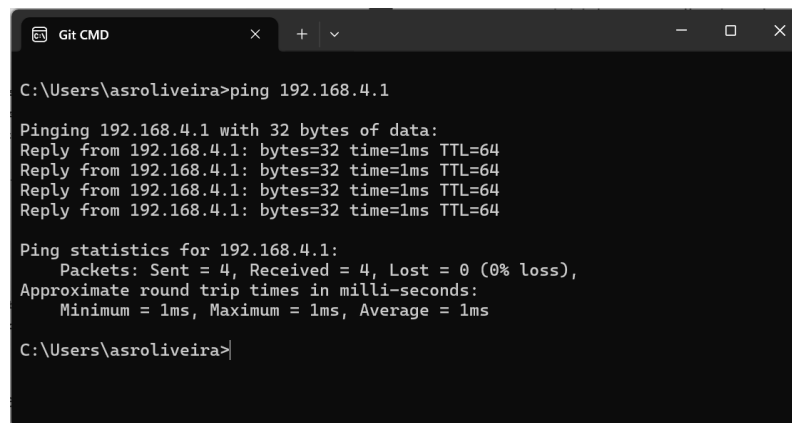
I (584) wifi:mode : softAP (10:51:db:01:3b:2d)
I (694) wifi:Total power save buffer number: 16
I (694) wifi:Init max length of beacon: 752/752
I (694) wifi:Init max length of beacon: 752/752
I (704) esp_netif_lwip: DHCP server started on interface WIFI_AP_DEF with IP: 192.168.4.1
I (704) wifi softAP: wifi_init_softap finished. SSID:AO-ESP32-AP password:12345678 channel:1
I (714) main_task: Returned from app_main()
```

At this stage, the ESP operates as an Access Point, ready to receive connections and assign DHCP leases.

Question 5: Using the laptop, select the Wi-Fi network created by the ESP32-C6 and connect with the defined password. You should see a log message in the console indicating the assigned IP address, as shown in the following listing:

```
I (171904) wifi:station: 00:72:ee:a8:53:4a join, AID=1, bgn, 40U
I (171934) wifi softAP: station 00:72:ee:a8:53:4a join, AID=1
I (172074) esp_netif_lwip: DHCP server assigned IP to a client, IP is: 192.168.4.2
```

Question 6: Using the ping command, test the connectivity with the Access Point. You should observe a result similar to that in Figure 1.



```
Git CMD
C:\Users\asroliveira>ping 192.168.4.1

Pinging 192.168.4.1 with 32 bytes of data:
Reply from 192.168.4.1: bytes=32 time=1ms TTL=64
Reply from 192.168.4.1: bytes=32 time=1ms TTL=64
Reply from 192.168.4.1: bytes=32 time=1ms TTL=64
Reply from 192.168.4.1: bytes=32 time=1ms TTL=64

Ping statistics for 192.168.4.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 1ms, Average = 1ms

C:\Users\asroliveira>
```

Figure 1 - Access Point connectivity test.

SoftAP mode is useful at various stages of an IoT device's lifecycle. First, it facilitates initial setup, allowing the user to connect directly to the device via their mobile phone to enter Wi-Fi network credentials or adjust essential parameters. This mode is equally advantageous in scenarios where the device needs to operate autonomously, without relying on an external router, such as in temporary installations, isolated industrial environments, or field measurements. In addition, it can be a very practical tool for diagnostics and maintenance, enabling the recovery of devices with connection failures, the changing of settings, or the performance of tests without resorting to cables or additional infrastructure. Finally, SoftAP allows providing local control interfaces, such as configuration web pages or dashboards, ensuring direct communication without the need for internet access.

3.3 Web Server

In this section, we will cover the *https_server/simple example* of the ESP-IDF platform for the ESP32-C6 SoC. This example demonstrates how to create a secure web server (HTTPS) running directly on the device. The program establishes a network connection (via Wi-Fi), loads a certificate and private key embedded in the firmware, and starts an HTTPS server that listens for requests on the secure port. Then, it registers a simple handler for the GET method on the route “/”, responding with a small HTML page. The example also shows how to start and stop the server automatically when the network interface gains or loses connectivity, illustrating the basic operation of the TLS stack, the secure HTTP server, and network event management on the ESP32-C6. For this exercise, you will need to configure your mobile phone as an Access Point so that it assigns an IP address to the ESP32-C6 SoC upon initialization.

Question 7: In VSCode, create a new project from the *https_server -> simple* template. Using the **menuconfig** command, define the SSID and Password of the Wi-Fi network created by your cell phone in the *Example Configuration* section. Compile the program and download the firmware for the SoC. You should observe the following event log in the console:

```
...
I (400) main_task: Started on CPU0
I (400) main_task: Calling app_main()
I (420) example_connect: Start example_connect.
...
W (590) wifi:ACK_TAB0 :0x 90a0b, QAM16:0x9 (24Mbps), QPSK:0xa (12Mbps), BPSK:0xb (6Mbps)
W (590) wifi:CTS_TAB0 :0x 90a0b, QAM16:0x9 (24Mbps), QPSK:0xa (12Mbps), BPSK:0xb (6Mbps)
W (600) wifi:(agc)0x600a7128:0xd212c9f0, min.avgNF:0xce->0xd2 (dB), RCalCount:0x12c,
min.RRssi:0x9f0(-97.00)
I (610) wifi:11ax coex: WDEVAX_PTIO(0x55777555), WDEVAX_PTII(0x00003377).

I (610) wifi:mode : sta (10:51:db:01:3b:2c)
I (620) wifi:enable tsf
I (620) example_connect: Connecting to ASRO-Avr-AP...
...
I (3240) wifi:state: init -> auth (0xb0)
I (3340) wifi:state: auth -> assoc (0x0)
...
I (3750) wifi:connected with ASRO-Avr-AP, aid = 20, channel 9, BW20, bssid = 1e:21:b4:6f:b9:c2
I (3750) wifi:security: WPA2-PSK, phy:ax, rssi:-26, cipher(pairwise:0x3, group:0x3), pmf:0,
I (3760) wifi:pm start, type: 1, twt_start:0
...
```

```

I (5420) example_connect: Got IPv6 event: Interface "example_netif_sta" address:
fe80:0000:0000:0000:1251:dbff:fe01:3b2c, type: ESP_IP6_ADDR_IS_LINK_LOCAL
I (5880) example: Starting server
I (5880) esp_https_server: Starting server
I (5880) esp_https_server: Server listening on port 443
I (5880) example: Registering URI handlers
I (5880) esp_netif_handlers: example_netif_sta ip: 192.168.68.58, mask: 255.255.252.0, gw:
192.168.68.1
I (5890) example_connect: Got IPv4 event: Interface "example_netif_sta" address: 192.168.68.58
I (5900) example_common: Connected to example_netif_sta
I (5900) example_common: - IPv4 address: 192.168.68.58,
I (5910) example_common: - IPv6 address: fe80:0000:0000:0000:1251:dbff:fe01:3b2c, type:
ESP_IP6_ADDR_IS_LINK_LOCAL
I (5920) main_task: Returned from app_main()

```

Currently, the basic server is active and has connected to the Access Point created on the cell phone. The AP assigned the IP address 192.168.68.58 to the ESP32-C6, which serves a web page on this same IP address (port 80, the default). Note that, depending on your AP configuration, the assigned IP address may (and should) be different.

Question 8: Connect your computer to the Access Point created by the cell phone and access the address 192.168.68.58. As you can see, the browser will display a security warning, preventing you from immediately viewing the page served by the ESP32-C6.

This happens because the certificate used by the server is self-signed — that is, it was not issued by a recognized certification authority (such as Let's Encrypt or DigiCert). Thus, the browser cannot confirm the server's identity and considers the connection potentially insecure.

This factor is usually accompanied by another: the name indicated in the certificate does not match the address used to access the device (for example, we are using <https://192.168.68.58/>, but the certificate lists a different name). In a development and testing context, this behavior is expected. The solution is simply to accept the security exception in the browser or, alternatively, generate a certificate with the correct name or install the issuing authority as trusted in the system.

For now, accept the security exception, and you should see the web page in Figure 2.

Question 9: Update the program to print the instantaneous temperature and humidity whenever a new page is requested.

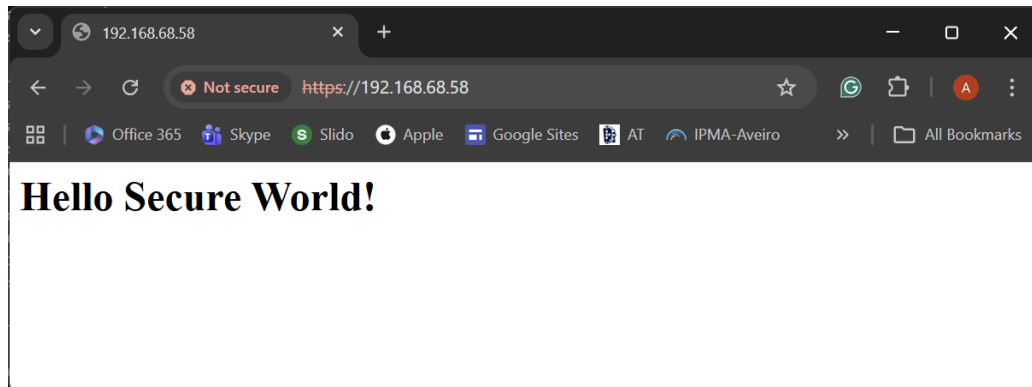


Figure 2 - Page obtained from the basic web server.

This example establishes a web server on the ESP32-C6 that operates as a Wi-Fi network client to obtain an IP address and have connectivity. This implies that anyone who wants to access the web server has to “discover” the IP address assigned to the ESP32-C6, which is not always easy. One way to address this problem is to establish that the web server simultaneously serves as an Access Point.

Question 10: Modify the program so that the web server simultaneously acts as a softAP, allowing a Wi-Fi client to connect, automatically assigning it an IP address, and serving it a web page with the current temperature and humidity whenever requested.